

OTIS 승강기 측정 앱 — 원시 센서 데이터 설명 자료

회의: 금요일 11:00 Zoom | 측정 기기: Galaxy S22

목적: 원시(raw) 센서 데이터와 핵심 코드 구조 공유

1. 한 줄 요약

화면의 8.2mg / 32.5mg = 필터·Aptp 적용 후 결과값

raw.txt = S22 센서로 수집한 원본 기록

원본을 남기면 필터/공식을 바꿔도 재탐승 없이 재분석 가능

2. 데이터 3종류

- ① 앱 결과값: 필터 + 정속 구간 + Aptp(A95)
- ② raw.txt: 센서 샘플 전체 (원본)
- ③ raw_summary.txt: ②의 min/max/mean/P-P 정리본

UI는 ①만 표시, 메일 첨부로는 ②·③ 전송

3. 전체 흐름

S22 센서 수집 (256Hz = 초당 256번)

→ raw.txt 저장

→ 진동: linear → 필터 → 정속 구간 → Aptp

→ 속도·거리: (raw $\times$ gravity) → 적분 → 속도 → 적분 → 거리

4. 핵심 코드

① 센서 수집 — `SensorStreamHandler.kt`

개발 용어: 256Hz = 초당 256번 샘플링. TYPE_* = Android 센서 종류.

```
when (event.sensor.type) {  
    Sensor.TYPE_ACCELEROMETER ->    // rawX/Y/Z (중력 포함)  
    Sensor.TYPE_GRAVITY ->          // gravityX/Y/Z  
    Sensor.TYPE_LINEAR_ACCELERATION -> // linearX/Y/Z (진동용)  
}  
// 256Hz로 리샘플 (초당 256샘플), 단위 mg
```

② 모션 성분 — `sensor_sample.dart`

거리/속도용: motion = raw $\times$ gravity (없으면 linear 사용)

```
double get motionZ =>  
    rawZ != null && gravityZ != null ? rawZ! - gravityZ! : z;
```

③ raw.txt 저장 — `parse_raw.dart writeEvimp1()`

EVIMP1 포맷으로 전체 샘플 직렬화 (오프라인 재분석용)

```
EVIMP1  
256  
# columns: tsUs linearX linearY linearZ noiseDb  
#          rawX rawY rawZ gravityX gravityY gravityZ  
(tsUs) (linear) (noise) (raw) (gravity)  
...
```

④ 파이프라인 분리 — `measurement_engine.dart`

진동 경로와 거리/속도 경로를 분리

```
// 거리/속도
motion = raw - gravity
integ = MotionIntegrator.integrate(motion)

// 진동
vibration = filter(linear)
→ 정속 구간에서 Aptp 산출
```

⑤ 속도·거리 적분 — motion_integrator.dart

가속도를 적분하여 속도, 속도를 적분하여 거리

```
accel[i] = motionZ[i] * mgToMetersPerSecondSquared; // m/s²

// 속도 적분
signedSpeed[i] = signedSpeed[i-1] + accel[i] * dt;

// 거리 적분
signedPos[i] = signedPos[i-1] + signedSpeed[i] * dt;

maxSpeed = max(|v|), distance = |s_end|
```

5. 개발 용어만 풀어 쓰기

- 256Hz → 초당 256번 샘플링
 - TYPE_ACCELEROMETER → 가속도계 원값 센서
 - TYPE_GRAVITY → 중력 추정 센서
 - TYPE_LINEAR_ACCELERATION → 중력 제외 가속도 센서
 - EventChannel / 리샘플 → Flutter로 넘기기 전 시간 간격 맞춤
- ※ 적분, Aptp, P-P, baseline 등은 측정/신호처리 용어 그대로 사용

6. 코드 파일 위치

- android/.../SensorStreamHandler.kt — 센서 수집
- lib/domain/measure/sensor_sample.dart — 샘플 모델 / motion
- lib/domain/parse_raw.dart — raw.txt I/O
- lib/domain/measure/measurement_engine.dart — 분석 오케스트레이션
- lib/domain/measure/motion_integrator.dart — 적분
- lib/domain/measure/signal_filters.dart — 진동 필터
- lib/domain/measure/vibration_metrics.dart — Aptp

7. 왜 이렇게 했나

- 결과만 남기면 수치 검증·재분석이 불가능
- 원본(raw)을 남기면 사무실에서 리플레이 가능
- Lift Check 비교는 A95끼리 같은 지표로
- 값을 깎아 맞추지 않고 계산 근거를 남김

8. 금요일 논의 포인트

- 원본 / 결과값 분리 구조 유지
- 다음: 동일 엘베 Lift Check 동시 측정 (A95 비율)
- 폰 거치 방향 통일 (X/Y 매핑)
- Z축 Aptp가 Lift Check보다 높은 원인 계속 분해

핵심: raw = 센서 원본 기록 / 화면 숫자 = 필터·Aptp 적용 결과. 원본을 남기고 검증하는 단계.